

Uwarunkowanie zadań i stabilność numeryczna algorytmów

Ewa Kasprzak

10 listopada 2024

1 Wstęp

Współczesne obliczenia numeryczne są nieodłącznie związane z problemem reprezentacji danych w komputerach oraz błędami, które pojawiają się w wyniku ograniczonej precyzji arytmetyki maszynowej. W szczególności, podczas rozwiązywania problemów matematycznych, błędy w danych wejściowych, takie jak zaokrąglenia liczb czy inne nieprecyzyjności, mogą prowadzić do istotnych zniekształceń wyników obliczeń. Z tego powodu jednym z kluczowych zagadnień w analizie algorytmów numerycznych jest ocena ich stabilności oraz uwarunkowania problemu, który jest rozwiązywany.

Wskaźnik uwarunkowania jest miarą, która określa, jak zmiany w danych wejściowych (np. błędy zaokrąglenia) wpływają na dokładność wyniku obliczeń. W przypadku dobrze uwarunkowanych problemów, niewielkie zmiany w danych wejściowych prowadzą do proporcjonalnych zmian w wynikach, co umożliwia uzyskanie dokładnych wyników mimo ograniczeń precyzyjnych. Z kolei problemy źle uwarunkowane charakteryzują się dużą wrażliwością na błędy w danych, co sprawia, że numeryczne rozwiązanie staje się trudne, a niewielkie zaburzenia w danych mogą prowadzić do znacznych odchyleń w wynikach.

Stabilność algorytmów numerycznych odnosi się do zdolności metod obliczeniowych do minimalizowania wpływu błędów zaokrąglenia na ostateczny wynik. Algorytmy stabilne numerycznie są zaprojektowane w taki sposób, aby kontrolować kumulację błędów, zapobiegając ich negatywnemu wpływowi na precyzję wyników. Natomiast algorytmy niestabilne mogą prowadzić do znacznych błędów, szczególnie w przypadkach, gdy problem jest źle uwarunkowany lub gdy w trakcie obliczeń dochodzi do operacji, które prowadzą do utraty istotnych cyfr znaczących. Dlatego przy projektowaniu algorytmów numerycznych, szczególną uwagę należy zwrócić na stabilność metod, tak aby zapewnić niezawodne wyniki w obliczeniach, które są narażone na błędy reprezentacji liczb.

Celem niniejszej pracy jest zbadanie zjawiska uwarunkowania zadań matematycznych oraz stabilności algorytmów numerycznych. Zostaną omówione teo-

retyczne podstawy tych zagadnień, przedstawione przykłady problemów o różnym stopniu uwarunkowania oraz zaprezentowane techniki numeryczne, które pozwalają na uzyskanie stabilnych wyników obliczeniowych.

2 Iloczyn skalarny

Aby zbadać uwarunkowanie zadania liczenia iloczynu skalarnego, kontynuowano eksperyment liczenia iloczynu skalarnego opisany w poprzednim sprawozdaniu, wprowadzając drobne zmiany w wektorach wejściowych. Zmiany polegały na usunięciu ostatniej cyfry (9) z wartości x_4 oraz ostatniej cyfry (7) z wartości x_5 , po czym ponownie obliczono iloczyn skalarny tych wektorów:

$$x = [2.718281828, -3.141592654, 1.414213562, 0.577215664, 0.301029995]$$

$$y = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]$$

Typ	W przód	W tył	Max -> Min	Min -> Max
Float32	-0.4999443	-0.4543457	-0.5	-0.5
Float64	-0.004296342739891585	-0.004296342998713953	-0.004296342842280865	-0.004296342842280865
Float64 (real)	-0.0042963432	-0.0042963432	-0.0042963432	-0.0042963432

Tabela 1: Porównanie sumowania iloczynów elementów wektorów w różnych kolejnościach.

Choć wprowadzone zmiany były minimalne (rzędu 10^{-10}), ich wpływ na wyniki był zauważalny. W szczególności w przypadku arytmetyki **Float64** wyniki uległy poprawie, a różnice względem rzeczywistej wartości występowały jedynie na 9. miejscu po przecinku. Może to sugerować, że zmiany te mieszczą się w granicach precyzji kalkulatora używanego do porównań, który mógł zaokrąglić wynik do tej właśnie liczby.

Zaskakującym odkryciem było to, że w przypadku arytmetyki **Float32** wprowadzone zmiany nie miały żadnego wpływu na wynik końcowy. Jest to prawdopodobnie wynikiem ograniczonej precyzji tej arytmetyki, która pozwala na reprezentowanie liczb z dokładnością do 7 cyfr po przecinku. Zatem zmiana na 10. miejscu po przecinku była zbyt mała, by wpłynęła na obliczenia, które pozostały identyczne z wcześniejszymi wynikami.

Obserwacje te podkreślają dużą wrażliwość obliczeń na niewielkie zmiany w danych wejściowych, co wskazuje na źle uwarunkowane zadanie. Drobne zmiany w danych wejściowych mogą prowadzić do znacznych różnic w wynikach końcowych, co jest charakterystyczne dla problemów numerycznych o niskiej stabilności, gdzie błędy zaokrągleń mogą się kumulować i wpływać na ostateczne wyniki.

3 Granica funkcji $f(x) = e^x \ln(1 + e^{-x})$

Funkcja $f(x) = e^x \ln(1 + e^{-x})$ jest interesującą kombinacją funkcji wykładniczej i logarytmicznej. Składa się z dwóch części: pierwsza, e^x , rośnie wykładniczo, natomiast druga część, $\ln(1 + e^{-x})$, ma charakter bardziej złożony, zbliżający się do logarytmu naturalnego dla małych wartości x i asymptotycznie dążący do wartości $\ln(1) = 0$ dla dużych x .

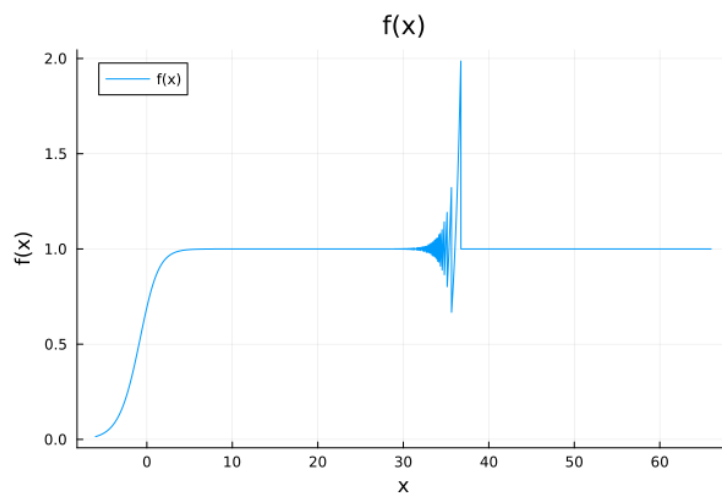
Zrozumienie tej funkcji jest kluczowe, ponieważ dla dużych wartości x wyrażenie e^{-x} staje się bardzo małe, co powoduje, że logarytmiczna część funkcji zbliża się do zera. Z kolei dla małych x (bliskich zera lub ujemnych) oba składniki mają znaczący wpływ na wartość funkcji, co sprawia, że jej zachowanie w różnych zakresach x jest interesujące do analizy.

W ramach tego zadania, celem jest narysowanie wykresu tej funkcji, aby zobaczyć, jak zmienia się jej postać w zależności od wartości x , a także obliczenie granicy funkcji w miarę jak x dąży do nieskończoności. Obliczenie tej granicy pozwala lepiej zrozumieć, do jakiej wartości funkcja dąży dla dużych x , co jest ważne, by móc porównać zachowanie funkcji z wynikami analitycznymi.

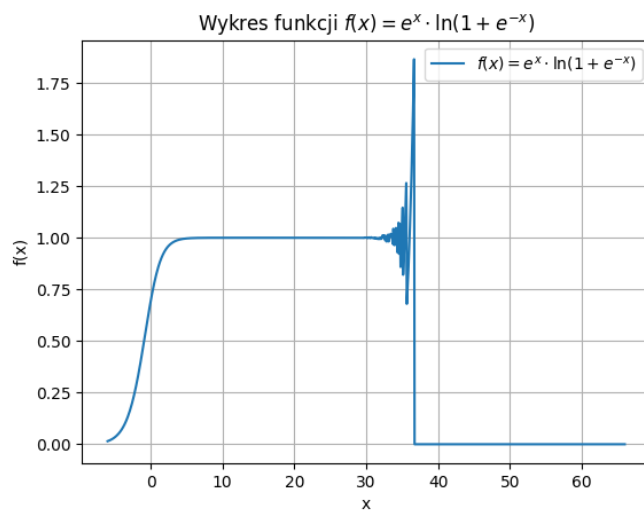
Wykresy funkcji zostaną wykonane przy użyciu dwóch różnych narzędzi do wizualizacji (Julia, Python), co pozwoli na porównanie wyników oraz lepsze zrozumienie zachowania funkcji. Na podstawie tych wykresów i obliczonej granicy będziemy mogli wyciągnąć wnioski dotyczące asymptotycznych właściwości funkcji $f(x)$ oraz omówić, jakie zjawiska mają miejsce, gdy x staje się bardzo duże.

3.1 Wizualizacja funkcji

Poniżej przedstawiono wykresy funkcji $f(x) = e^x \ln(1 + e^{-x})$ wykonane przy użyciu dwóch różnych narzędzi do wizualizacji.



Rysunek 1: Wykres funkcji stworzony w Julii

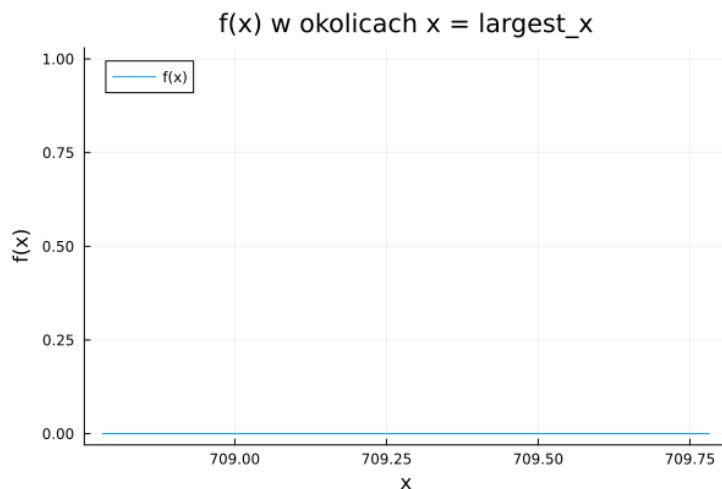


Rysunek 2: Wykres funkcji stworzony w Pythonie

3.2 Wnioski

Wykresy narysowane w językach Julia i Python sugerują, że granica funkcji $\lim_{x \rightarrow \infty} f(x)$ wynosi 0. Dodatkowo, dla największej wartości x , dla której udało się obliczyć wartość funkcji w Julii, wynoszącej około 709.79, wartość funkcji

wynosi 0.0.



Rysunek 3: Wykres funkcji w pobliżu 709.79

Ograniczenie na największą wartość, dla której jesteśmy w stanie obliczyć wartość funkcji w Julii, wynika z ograniczeń reprezentacji liczb zmiennoprzecinkowych w komputerach. Dla wartości x większych niż około 709.79, wyrażenie e^x staje się na tyle duże, że jest traktowane jako nieskończoność (Inf). Z kolei e^{-x} dla takich dużych x jest na tyle małe, że jest zaokrąglane do zera. W efekcie, w funkcji

$$f(x) = e^x \ln(1 + e^{-x})$$

otrzymujemy wyrażenie

$$\text{Inf} \times \ln(1)$$

co daje

$$\text{Inf} \times 0$$

a takie wyrażenie jest matematycznie nieokreślone. W wyniku tego obliczenia, Julia zwraca wartość NaN (Not a Number), wskazując na problem z obliczeniami w tym zakresie.

Mimo, że wykresy wskazują na granicę równą 0, faktyczną granicą jest 1.

Aby policzyć granicę funkcji $f(x) = e^x \cdot \ln(1 + e^{-x})$ dla $x \rightarrow \infty$, przeanalizujemy krok po kroku zachowanie obu składników funkcji.

Funkcję można rozłożyć na:

$$f(x) = e^x \cdot \ln(1 + e^{-x})$$

i rozpatrzmy osobno zachowanie wyrażeń e^x oraz $\ln(1 + e^{-x})$ dla $x \rightarrow \infty$.

Krok 1: Analiza $\ln(1 + e^{-x})$ dla $x \rightarrow \infty$

Gdy $x \rightarrow \infty$, wyrażenie e^{-x} dąży do zera, ponieważ e^{-x} maleje wykładniczo do zera wraz ze wzrostem x . Wówczas:

$$\ln(1 + e^{-x}) \approx \ln(1 + 0) = \ln(1) = 0$$

Zatem dla bardzo dużych wartości x :

$$\ln(1 + e^{-x}) \approx 0$$

Krok 2: Analiza e^x dla $x \rightarrow \infty$

Jednocześnie, gdy $x \rightarrow \infty$, wyrażenie e^x rośnie wykładniczo do nieskończoności:

$$e^x \rightarrow \infty$$

Krok 3: Zastosowanie iloczynu obu wyrażeń

Teraz mamy:

$$f(x) = e^x \cdot \ln(1 + e^{-x})$$

Dla $x \rightarrow \infty$ otrzymujemy iloczyn e^x (dążącego do nieskończoności) i $\ln(1 + e^{-x})$ (dążącego do zera). Jest to sytuacja typu $\infty \cdot 0$, więc użyjemy przekształcenia, aby wyrazić funkcję w postaci ilorazu.

Krok 4: Przekształcenie funkcji do postaci ilorazu

Przekształćmy $f(x)$ w taki sposób, aby móc zastosować regułę de l'Hospitala:

$$f(x) = \frac{\ln(1 + e^{-x})}{e^{-x}}$$

Gdy $x \rightarrow \infty$:

- Licznik $\ln(1 + e^{-x}) \rightarrow 0$
- Mianownik $e^{-x} \rightarrow 0$

Otrzymujemy formę nieoznaczoną $\frac{0}{0}$, więc możemy zastosować regułę de l'Hospitala.

Krok 5: Zastosowanie reguły de l'Hospitala

Zaczynamy od pochodnych licznika i mianownika:

1. Pochodna licznika: $\frac{d}{dx} \ln(1 + e^{-x}) = \frac{-e^{-x}}{1 + e^{-x}}$
2. Pochodna mianownika: $\frac{d}{dx} e^{-x} = -e^{-x}$

Teraz możemy zapisać:

$$f(x) = \lim_{x \rightarrow \infty} \frac{\ln(1 + e^{-x})}{e^{-x}} = \lim_{x \rightarrow \infty} \frac{-e^{-x}}{1 + e^{-x}} \cdot \frac{1}{-e^{-x}}$$

Po uproszczeniu pozostaje:

$$f(x) = \lim_{x \rightarrow \infty} \frac{1}{1 + e^{-x}}$$

Krok 6: Obliczenie granicy

Gdy $x \rightarrow \infty$, wyrażenie e^{-x} dąży do zera, więc $1 + e^{-x} \rightarrow 1$. Wówczas:

$$f(x) = \frac{1}{1 + 0} = 1$$

Ostateczna odpowiedź

$$\lim_{x \rightarrow \infty} e^x \cdot \ln(1 + e^{-x}) = 1$$

Zatem granica funkcji $f(x) = e^x \cdot \ln(1 + e^{-x})$ dla $x \rightarrow \infty$ wynosi 1.

Taki sam wynik, czyli wartość 1 dają języki Python i Wolfram Alpha.

Podsumowując, algorytm obliczania wartości funkcji $f(x) = e^x \ln(1 + e^{-x})$ wykazuje niestabilność numeryczną. Małe błędy, które wynikają z zaokrągleń liczb zmiennoprzecinkowych w komputerze, mają znaczny wpływ na wynik końcowy.

3.3 Możliwa poprawa algorytmu

Dla dużych wartości x , gdzie e^{-x} staje się bardzo małe, wartość $\ln(1 + e^{-x})$ można przybliżyć jako e^{-x} , co minimalizuje wpływ błędów zaokrągleń i poprawia stabilność numeryczną. Zastosowanie tego przybliżenia jest uzasadnione przez rozwinięcie w szereg Taylora logarytmu w punkcie $y = 1$, co pozwala uprościć obliczenia dla dużych wartości x .

Krok 1: Rozwinięcie $\ln(1 + y)$ w szereg Taylora dla $y \approx 0$

Gdy $y = e^{-x}$ jest bardzo małe (dla dużych x), możemy skorzystać z rozwinięcia Taylora logarytmu naturalnego wokół $y = 0$:

$$\ln(1 + y) = y - \frac{y^2}{2} + \frac{y^3}{3} - \dots$$

Dla wartości $y \approx 0$, wyższe potęgi y stają się zaniedbywalne, co pozwala przybliżyć $\ln(1 + y)$ jako y :

$$\ln(1 + y) \approx y.$$

Krok 2: Zastosowanie przybliżenia do $\ln(1 + e^{-x})$

Dzięki rozwinięciu w szereg Taylora możemy przyjąć:

$$\ln(1 + e^{-x}) \approx e^{-x}.$$

Dla funkcji $f(x) = e^x \ln(1 + e^{-x})$ uzyskujemy więc przybliżenie:

$$f(x) \approx e^x \cdot e^{-x} = 1.$$

Krok 3: Określenie granicy x , przy której przybliżenie staje się konieczne

Błędy zaokrągleń wynikające z ograniczonej precyzji liczb zmiennoprzecinkowych (np. Float64) mogą wpływać na wynik obliczeń. W przypadku precyzji podwójnej (Float64), maszynowy epsilon wynosi około 2.22×10^{-16} . Gdy e^{-x} staje się mniejsze od tej wartości, $1 + e^{-x}$ zaokrągla się do 1, co powoduje stratę znaczenia w wyniku $\ln(1 + e^{-x})$.

Aby wyznaczyć wartość x , dla której $e^{-x} \approx 2.22 \times 10^{-16}$, rozważamy równanie:

$$e^{-x} = 2.22 \times 10^{-16}$$

i bierzemy logarytm obu stron:

$$x \approx -\ln(2.22 \times 10^{-16}) \approx 36.04.$$

Eksperymentalne wyznaczenie wartości x

Aby dokładnie określić wartość x , dla której $f(x) = 0$, zastosowano algorytm w języku Julia, który wyszukuje najmniejszą taką wartość x . Wynik uzyskany przez algorytm to:

$$x = 36.736800577044484$$

Oznacza to, że wartość x , dla której $f(x) = 0$, leży w przedziale:

$$(36.736800577044484 - 2^{-26}, 36.736800577044484 + 2^{-26}).$$

Przedział ten wynika z implementacji algorytmu, który określa dokładność w granicach 2^{-26} wokół wyznaczonego x .

Różnica między uzyskaną wartością x a oczekiwaną teoretyczną wynika najprawdopodobniej z błędów numerycznych powstających przy obliczaniu wyrażenia $1 + e^{-x}$, które stanowi argument logarytmu w funkcji $f(x) = e^x \ln(1 + e^{-x})$.

Podsumowanie algorytmu

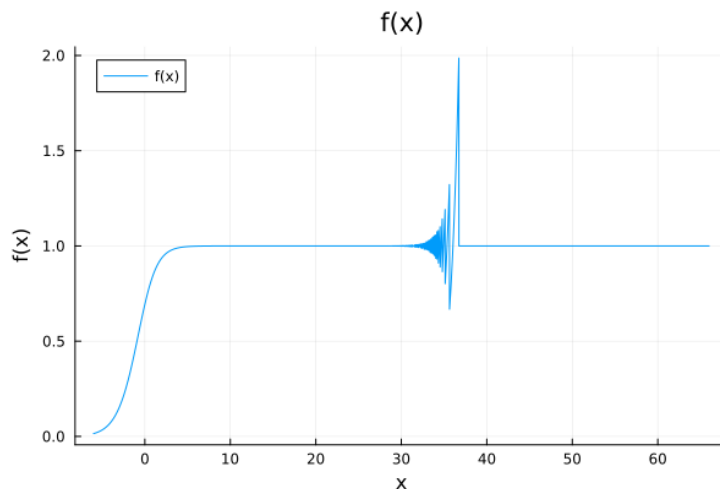
Na podstawie wyznaczonych wartości, dla $x \geq 36.736800577044484$ zalecane jest stosowanie przybliżenia:

$$f(x) \approx 1,$$

aby uniknąć błędów wynikających z zaokrągleń. Takie podejście stabilizuje obliczenia i zapewnia dokładność dla dużych wartości x , gdy bezpośrednie obliczenie $\ln(1 + e^{-x})$ mogłoby prowadzić do niestabilnych wyników.

```
function f(x)
    if Float64(x) < 36.736800577044484
        return e^x * log(1 + e^(-x))
    else
        return 1
    end
end
```

Efekty zastosowania pokazane są na wykresie 4.



Rysunek 4: Wykres usprawnionej funkcji stworzony w Julii

4 Wskaźnik uwarunkowania macierzy

Wskaźnik uwarunkowania macierzy, często oznaczany jako $\text{cond}(A)$, jest kluczowym pojęciem w analizie numerycznej, które pomaga ocenić, jak stabilne są rozwiązania układów równań z macierzą A względem małych zaburzeń w danych wejściowych, takich jak elementy macierzy A lub wektora b .

Poniżej znajduje się opis kilku wybranych pojęć związanych z wskaźnikiem uwarunkowania macierzy.

- **Wartość wskaźnika uwarunkowania:**

Wskaźnik uwarunkowania macierzy $\text{cond}(A)$ jest zawsze większy lub równy

1, co wynika z definicji normy i własności odwrotności macierzy.

- **Macierz dobrze uwarunkowana ($\text{cond}(A) \approx 1$):**

Gdy wskaźnik uwarunkowania macierzy jest bliski 1, macierz uznaje się za dobrze uwarunkowaną. W przypadku takiej macierzy niewielkie błędy w elementach A lub b powodują jedynie minimalne błędy w rozwiązaniu x . Algorytmy numeryczne są w stanie znaleźć rozwiązanie o wysokiej dokładności, ograniczonej jedynie przez drobne błędy zaokrągleń.

- **Macierz źle uwarunkowana ($\text{cond}(A) \gg 1$):**

Gdy wskaźnik uwarunkowania jest znacznie większy od 1, nawet niewielkie zaburzenia w macierzy A lub wektorze b mogą prowadzić do dużych zmian w wyniku x . Macierze źle uwarunkowane są problematyczne dla algorytmów numerycznych, ponieważ błędy numeryczne mogą znacząco wpływać na uzyskane rozwiązanie.

- **Macierz nieodwracalna ($\text{cond}(A) \rightarrow \infty$):**

Jeśli wskaźnik uwarunkowania dąży do nieskończoności, macierz A staje się nieodwracalna. W takim przypadku niemożliwe jest jednoznaczne wyznaczenie rozwiązania układu $Ax = b$.

- **Znaczenie wskaźnika uwarunkowania:**

Wskaźnik uwarunkowania jest istotny, ponieważ pozwala przewidzieć wpływ błędów zaokrągleń na rozwiązanie. W przypadku dużej wartości $\text{cond}(A)$, błąd popełniony przy wyliczaniu $b = Ax_{\text{exact}}$ będzie się propagował do wyniku x . Aby uzyskać dokładne rozwiązanie x , potrzebujemy zatem bardzo dokładnej wartości b .

- **Wektor błędu i wektor residualny:**

W celu dokładniejszego opisu wpływu zaburzeń można wprowadzić wektory błędu i residualny:

- **Wektor błędu:** $e = x - x'$, gdzie x jest dokładnym rozwiązaniem, a x' jest rozwiązaniem przybliżonym.
- **Wektor residualny:** $r = b - Ax'$, który pokazuje różnicę między wektorem prawym b a wynikiem uzyskanym z przybliżonego rozwiązania.
- Zgodnie z równaniem $Ae = r$, wektor błędu e jest powiązany z wektorem residualnym r poprzez macierz A .

4.1 Macierz Hilberta

Macierz Hilberta jest klasycznym przykładem macierzy źle uwarunkowanej, co oznacza, że nawet małe zaburzenia w danych wejściowych mogą prowadzić do dużych błędów w rozwiązaniach numerycznych układów równań, które ją wykorzystują.

4.1.1 Definicja macierzy Hilberta

Macierz Hilberta H to macierz o wymiarach $n \times n$, której elementy H_{ij} są zdefiniowane jako:

$$H_{ij} = \frac{1}{i + j - 1},$$

gdzie $i, j = 1, 2, \dots, n$.

Dla przykładu, dla $n = 3$ macierz Hilberta przyjmuje postać:

$$H = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}.$$

4.1.2 Właściwości macierzy Hilberta

Macierz Hilberta ma kilka szczególnych właściwości:

- **Symetryczność:** Macierz Hilberta jest symetryczna, co oznacza, że $H = H^T$.
- **Dodatnio określona:** Każda macierz Hilberta jest dodatnio określona, co gwarantuje, że wszystkie jej wartości własne są dodatnie.
- **Źle uwarunkowana:** Wskaźnik uwarunkowania macierzy Hilberta rośnie szybko wraz z wymiarem n . Oznacza to, że dla dużych wartości n macierz Hilberta jest szczególnie podatna na błędy numeryczne. Nawet niewielkie zaburzenia elementów macierzy lub wektora b mogą prowadzić do dużych błędów w rozwiązaniu układu $Hx = b$.

4.1.3 Eksperymentalne zbadanie własności macierzy Hilberta

W celu eksperymentalnego zbadania własności macierzy Hilberta zaimplementowano algorytm tworzący takową macierz.

```
function hilb(n::Int)
    if n < 1
        error("size n should be >= 1")
    end
    return [1 / (i + j - 1) for i in 1:n, j in 1:n]
end
```

Zaimplementowano również dwie metody rozwiązywania układów równań pierwszego stopnia

1. Metoda eliminacji Gaussa

```
function gauss(A, b)
    return A \ b
end
```

2. Metoda inwersji

```
function inversion(A, b)
    return inv(A) * b
end
```

Przetestowano następnie algorytmy na macierzach Hilberta od $n = 2, 3, \dots, 20$.
Otrzymane dane przedstawiono w tabeli 2.

n	cond	error gauss	error inv
2	19.28147006790397	5.661048867003676e-16	1.4043333874306803e-15
3	524.0567775860644	8.022593772267726e-15	0.0
4	15513.73873892924	4.137409622430382e-14	0.0
5	476607.2502425855	1.6828426299227195e-12	3.3544360584359632e-12
6	1.4951058642254734e7	2.618913302311624e-10	2.0163759404347654e-10
7	4.753673567446793e8	1.2606867224171548e-8	4.713280397232037e-9
8	1.5257575538060041e10	6.124089555723088e-8	3.07748390309622e-7
9	4.9315375594102344e11	3.8751634185032475e-6	4.541268303176643e-6
10	1.602441698742836e13	8.67039023709691e-5	0.0002501493411824886
11	5.222701316549833e14	0.00015827808158590435	0.007618304284315809
12	1.7515952300879806e16	0.13396208372085344	0.258994120804705
13	3.1883950689209334e18	0.11039701117868264	5.331275639426837
14	6.200786281355982e17	1.4554087127659643	8.71499275104814
15	3.67568286586649e17	4.696668350857427	7.344641453111494
16	7.046389953630175e17	54.15518954564602	29.84884207073541
17	1.249010044779401e18	13.707236683836307	10.516942378369349
18	2.2477642911280653e18	10.257619124632317	24.762070989128866
19	6.472700911391398e18	102.15983486270827	109.94550732878284
20	1.1484020388436145e18	108.31777346206205	114.34403152557572

Tabela 2: Porównanie wyników dla macierzy Hilberta: błędy dla metody Gaussa i odwrotności

4.1.4 Wnioski

Wyniki wskazują na to, że różnica w błędach między metodą Gaussa a metodą odwrotności jest stosunkowo mała, zwłaszcza w przypadku mniejszych rozmiarów macierzy (np. $n = 2$ do $n = 5$).

- **Błędy dla małych n :** Wartości błędów w obu metodach są bardzo zbliżone, zwłaszcza dla małych rozmiarów macierzy (np. $n = 2, 3, 4$). Różnice w błędach są na poziomie 10^{-15} do 10^{-12} , co sugeruje, że dla takich rozmiarów macierzy obie metody działają równie dobrze.
- **Błędy dla większych n :** Wraz ze wzrostem rozmiaru macierzy (np. dla $n \geq 10$), błędy zaczynają rosnąć, ale nadal w przypadku obu metod błędy

są podobne. Dla dużych n (np. $n = 20$) błędy są znacznie większe, ale różnice między metodami są nadal niewielkie.

- **Ogólna tendencja:** Widać, że w przypadku większych macierzy (np. dla $n = 16$ do $n = 20$), zarówno metoda Gaussa, jak i metoda odwrotności, zaczynają generować większe błędy, co jest wynikiem coraz gorszego uwarunkowania macierzy Hilberta, co powoduje wzrost błędów numerycznych. Jednakże różnice między obiema metodami są wciąż na poziomie kilku jednostek, co sugeruje, że obie metody mają podobną stabilność numeryczną w tym przypadku.

4.1.5 Tempo wzrostu błędów numerycznych dla macierzy Hilberta

W celu wyznaczenia tempa wzrostu błędów numerycznych przy rozwiązywaniu układu równań $Ax = b$, gdzie A jest macierzą Hilberta, zebrano dane o błędach dla macierzy o wymiarze $n = 2, 3, \dots, 100$. Następnie użyto regresji liniowej w języku Python, aby dopasować modele wykładniczy oraz wielomianowy do rozkładu tych błędów. Zebrane dane dotyczące błędów zostały poddane logarytmowaniu w celu wyizolowania tempa wzrostu, a następnie przeprowadzono regresję wykładniczą i wielomianową w celu uzyskania przybliżonych równań opisujących ten wzrost.

Regresja wykładnicza

Regresja wykładnicza przyjmuje postać

$$y = a \cdot e^{b \cdot x},$$

gdzie x to rozmiar macierzy n , a y to logarytm błędu. Modele wykładnicze pozwalają uzyskać prostą i interpretowalną funkcję, która dobrze aproksymuje wzrost błędu w przypadku większych macierzy Hilberta.

Regresja wielomianowa

Zastosowano modele wielomianowe o różnych stopniach (od 2 do 9), aby dopasować krzywe do danych. Dzięki tej metodzie można uzyskać bardziej elastyczne modele, które mogą lepiej odwzorować złożone zależności między błędami a rozmiarem macierzy.

Optymalizacja dopasowania

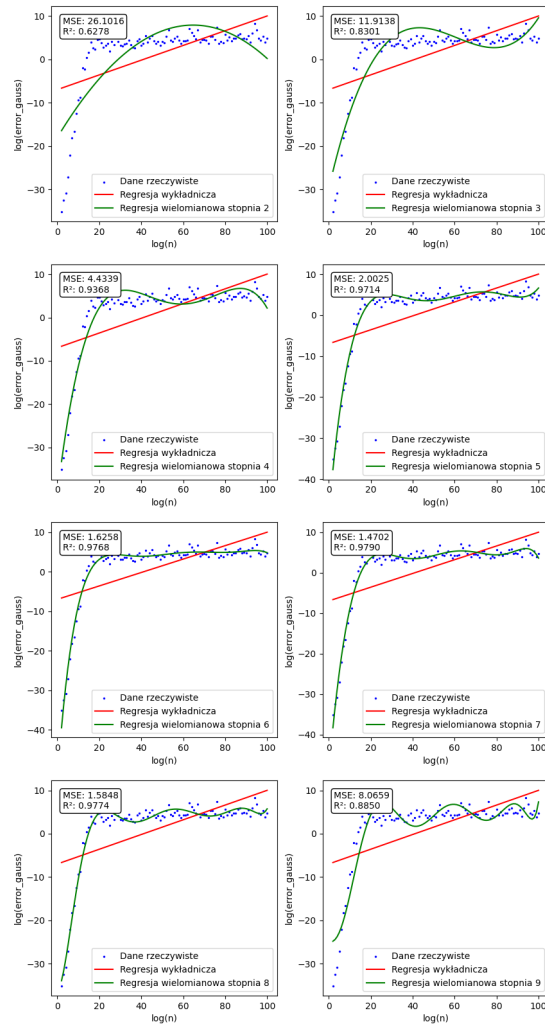
Aby uzyskać jak najlepsze dopasowanie modeli do danych, monitorowano dwa kluczowe wskaźniki: średnią (mean) oraz współczynnik determinacji R^2 . Celem było uzyskanie jak najbliższych wartości 1 dla obu tych wskaźników, co oznaczało jak najlepsze dopasowanie wykresów do rzeczywistych danych.

- **Średnia (mean):** Mierzono średnią różnicę między wartościami rzeczywistymi a przewidywanymi przez model, dążąc do minimalizacji tej różnicy.

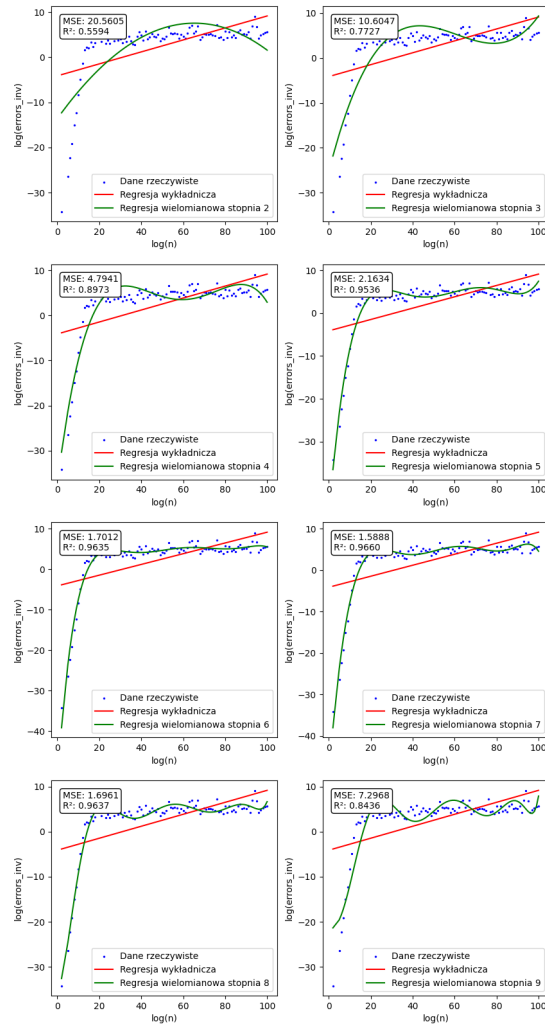
- **Współczynnik determinacji R^2 :** Używany do oceny jakości dopasowania modelu, gdzie wartość bliska 1 wskazuje na bardzo dobre dopasowanie.

Dzięki monitorowaniu tych wskaźników wybierano najlepszy model, który najlepiej aproksymował zbierane dane o błędach numerycznych. Wykorzystano pakiety `numpy`, `matplotlib`, `sklearn.linear_model` oraz `sklearn.preprocessing` z języka Python.

Na rysunkach 5 oraz 6 przedstawiono wynik działania programu dla błędów powstałych przy rozwiązywaniu $Ax = b$ metodą Gaussa i metodą inwersji.

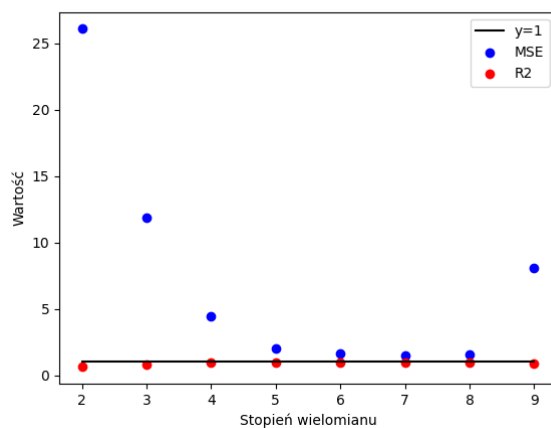


Rysunek 5: Wykres dla metody Gaussa

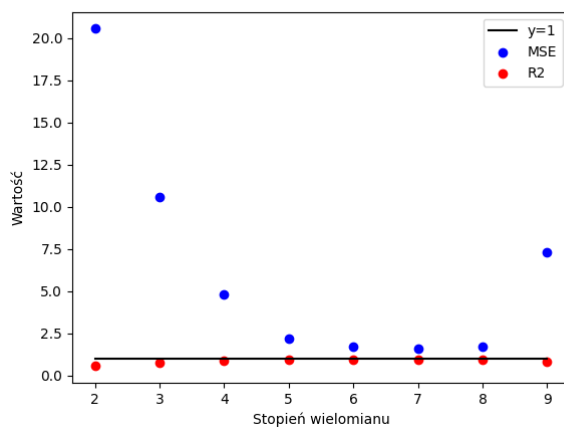


Rysunek 6: Wykres dla metody inwersji

W wyniku porównania wartości mean i R^2 otrzymano, że funkcją, która najlepiej opisuje tempo wzrostu błędów numerycznych na przedziale $n = 2, 3, \dots, 100$ jest wielomian 7 stopnia.



Rysunek 7: Porównanie średniej i R^2 dla metody Gaussa



Rysunek 8: Porównanie średniej i R^2 dla metody inwersji

4.1.6 Porównanie wyników z tempem wzrostu współczynnika uwarunkowania

W analizie porównawczej między błędami numerycznymi a współczynnikiem uwarunkowania macierzy Hilberta zauważono, że błędy numeryczne i współ-

czynnik uwarunkowania są proporcjonalne do siebie.

Wskaźnik uwarunkowania macierzy Hilberta H_N o rozmiarze N rośnie w sposób bardzo szybki w miarę zwiększania rozmiaru macierzy. Zgodnie z matematyczną analizą, jego wartość można wyrazić wzorem:

$$\text{cond}(H_N) = \frac{O(e^{3.5255N})}{\sqrt{N}}$$

Ten wynik oznacza, że wskaźnik uwarunkowania $\text{cond}(H_N)$ rośnie wykładniczo w zależności od rozmiaru macierzy N , co sprawia, że w przypadku dużych macierzy Hilberta, problem numeryczny staje się coraz trudniejszy do rozwiązania.

Choć wyrażenie zawiera pierwiastek kwadratowy w mianowniku, jego wpływ na tempo wzrostu jest minimalny w porównaniu do dominującego czynnika wykładniczego $e^{3.5255N}$. To właśnie ten składnik decyduje o wykładniczym charakterze wzrostu wskaźnika uwarunkowania.

Rozbieżność między danymi uzyskanymi podczas regresji oraz porównując do współczynnika uwarunkowania wynikają z niewystarczająco dużej próby danych użytych do regresji.

4.2 Macierz losowa

Macierzą losową o wymiarze n oraz współczynniku uwarunkowania c nazwiemy macierz utworzoną w wyniku działania algorytmu

```
function matcond(n::Int, c::Float64)
    if n < 2
        error("size n should be > 1")
    end
    if c < 1.0
        error("condition number c of a matrix should be >= 1.0")
    end
    (U,S,V)=svd(rand(n,n))
    return U*diagm(0 =>[LinRange(1.0,c,n);])*V'
end
```

4.2.1 Eksperymentalne badanie macierzy losowych

W ramach przeprowadzonych badań, testowano algorytmy rozwiązywania układów równań $Ax = b$ przy użyciu macierzy losowych. Porównano błędy numeryczne uzyskane dla dwóch metod: eliminacji Gaussa oraz inwersji macierzy.

n	cond	error
5	1.0	2.2752801345137457e-16
10	1.0	1.8242825833464351e-16
20	1.0	4.454748269764082e-16
5	10.0	2.3288234633381846e-16
10	10.0	2.1065000811460205e-16
20	10.0	5.720611996546166e-16
5	1000.0	1.2280090504320872e-14
10	1000.0	9.831086004686707e-15
20	1000.0	1.2320049408907698e-14
5	1.0e7	4.965068306494546e-17
10	1.0e7	9.495565506579851e-11
20	1.0e7	3.1659582588345563e-11
5	1.0e12	4.544524293818779e-5
10	1.0e12	1.513877465183363e-5
20	1.0e12	4.5669789048585326e-5
5	1.0e16	0.24423010776116366
10	1.0e16	0.04466364070761436
20	1.0e16	0.20690766061042995

Tabela 3: Matcond gauss

n	cond	error
5	1.0	1.5700924586837752e-16
10	1.0	2.220446049250313e-16
20	1.0	3.7072140595260095e-16
5	10.0	1.2161883888976234e-16
10	10.0	2.8522145930998397e-16
20	10.0	6.49741366860447e-16
5	1000.0	1.3528330643921842e-14
10	1000.0	8.204240168229037e-15
20	1000.0	1.9895989601924898e-14
5	1.0e7	5.531259355230536e-10
10	1.0e7	5.999908354159067e-11
20	1.0e7	1.686861377583355e-11
5	1.0e12	2.014248996819949e-6
10	1.0e12	4.400150362356051e-5
20	1.0e12	1.828127995656307e-5
5	1.0e16	0.13679685269131742
10	1.0e16	0.07485337229377177
20	1.0e16	0.32472516690425474

Tabela 4: Matcond inv

4.2.2 Wnioski

Z wyników eksperymentu wynika, że dla macierzy losowych nie występują istotne różnice w poziomie błędów między obiema metodami. Dla tych samych wartości współczynnika uwarunkowania, błędy uzyskane dla różnych rozmiarów macierzy były porównywalne. Wskazuje to na fakt, że obie metody charakteryzują się podobną stabilnością numeryczną.

5 Wielomian Wilkinsona

Wielomian Wilkinsona to wielomian stopnia 20, który ma pierwiastki w miejscach $x = 1, 2, \dots, 20$. Pierwiastki te są dosyć odległe od siebie, ale mimo tego wielomian jest bardzo źle uwarunkowany. Z tego powodu, nawet niewielkie zmiany w jego współczynnikach mogą prowadzić do znacznych zmian w położeniu pierwiastków.

5.1 Rozwinięcie wielomianu Wilkinsona

Po rozpisaniu wielomianu Wilkinsona otrzymujemy współczynniki o następującej postaci:

$$\omega(x) = x^{20} - 210x^{19} + 20615x^{18} - 1256850x^{17} + 53327946x^{16} - \dots + 2432902008176640000$$

5.2 Zaburzenie współczynnika a_{19}

Jeśli współczynnik przy x^{19} zostanie zmniejszony o 2^{-23} , to wpływ na wartość wielomianu będzie bardzo zauważalny.

Wilkinson wybrał perturbację 2^{-23} , ponieważ w komputerze Pilot ACE przy 30-bitowej precyzji zmiana ta odpowiadała błędowi w pierwszej pozycji bitowej, co miało znaczący wpływ na obliczenia.

5.3 Rozwinięcie w szereg Taylora

Aby zbadać, jak perturbacja wpływa na pierwiastek r , rozwijamy $\tilde{\omega}(x)$ w szereg Taylora wokół r .

$$\tilde{\omega}(r+h) = \omega(r) + h\omega'(r) + \frac{h^2}{2}\omega''(r) - \epsilon \left(z(r) + hz'(r) + \frac{h^2}{2}z''(r) \right) = 0.$$

Ponieważ $\omega(r) = 0$ (bo r jest pierwiastkiem), otrzymujemy:

$$h\omega'(r) + \frac{h^2}{2}\omega''(r) - \epsilon \left(z(r) + hz'(r) + \frac{h^2}{2}z''(r) \right) = 0.$$

Zakładając, że h jest małe (pierwiastek $r + h$ jest bliski pierwiastkowi r), dominujący wpływ na rozwiązanie mają pierwsze człony w równaniu. Ostatecznie, dla małych perturbacji, możemy zaniedbać wyrazy drugiego rzędu i otrzymać przybliżenie:

$$h \approx \frac{\epsilon z(r)}{\omega'(r)}.$$

Jeżeli pochodna $\omega'(r)$ jest mała, pierwiastki są stabilne, natomiast gdy pochodna jest duża, pierwiastki stają się niestabilne.

5.4 Analiza wpływu zaburzenia na pierwiastek $r = 20$

W kontekście wielomianu Wilkinsona, mamy do czynienia z wielomianem stopnia 20, którego pierwiastki to $r = 1, 2, \dots, 20$. Przykład dotyczy pierwiastka $r = 20$.

Wielomian Wilkinsona jest postaci:

$$\omega(x) = \prod_{i=1}^{20} (x - i),$$

a jego pochodna to:

$$\omega'(x) = \sum_{i=1}^{20} \prod_{j \neq i} (x - j).$$

W przypadku $r = 20$:

$$\omega'(r) = 19!.$$

Teraz, wprowadzamy zaburzenie $\epsilon = 2^{-23}$ i analizujemy wpływ na pierwiastek $r = 20$. Zaburzenie $z(x) = x^{19}$ w tym przypadku daje:

$$h \approx \frac{\epsilon z(20)}{\omega'(20)} = \frac{\epsilon \cdot 20^{19}}{19!}.$$

Podstawiając wartość $\epsilon = 2^{-23}$, otrzymujemy:

$$h \approx \frac{2^{-23} \cdot 20^{19}}{19!} \approx \epsilon \cdot 10^8 \approx 102.76.$$

5.5 Eksperymentalne badanie właściwości wielomianu Wilkinsona

Aby dokładniej zbadać problem, w języku programowania Julia zaimplementowano algorytmy, które umożliwiają szczegółową analizę właściwości wielomianu Wilkinsona.

Za pomocą funkcji `roots` w języku Julia wyznaczono pierwiastki wielomianu w jego postaci naturalnej, $|P(z_k)|$. Następnie, dla tych pierwiastków, obliczono wartość wielomianu w postaci iloczynowej, $|p(z_k)|$, oraz przeanalizowano różnicę między faktyczną a obliczoną wartością wielomianu, $|z_k - k|$. Wyniki eksperymentu przedstawiono w tabeli 5.

k	z_k	$ P(z_k) $	$ p(z_k) $	$ z_k - k $
1	0.9999999999996989	35696.50964788257	36626.425482422805	3.0109248427834245e-13
2	2.0000000000283182	176252.60026668405	181303.93367257662	2.8318236644508943e-11
3	2.9999999995920965	279157.6968824087	290172.2858891686	4.0790348876384996e-10
4	3.9999999837375317	3.0271092988991085e6	2.0415372902750901e6	1.626246826091915e-8
5	5.000000665769791	2.2917473756567076e7	2.0894625006962188e7	6.657697912970661e-7
6	5.999989245824773	1.2902417284205095e8	1.1250484577562995e8	1.0754175226779239e-5
7	7.000102002793008	4.805112754602064e8	4.572908642730946e8	0.00010200279300764947
8	7.999355829607762	1.6379520218961136e9	1.5556459377357383e9	0.0006441703922384079
9	9.002915294362053	4.877071372550003e9	4.687816175648389e9	0.002915294362052734
10	9.990413042481725	1.3638638195458128e10	1.2634601896949205e10	0.009586957518274986
11	11.02502932909318	3.585631295130865e10	3.300128474498415e10	0.025022932909317674
12	11.953283253846857	7.533332360358197e10	7.388525665404988e10	0.04671674615314281
13	13.07431403244734	1.9605988124330817e11	1.8476215093144193e11	0.07431403244734014
14	13.914755591802127	3.5751347823104315e11	3.5514277528420844e11	0.08524440819787316
15	15.075493799699476	8.21627123645597e11	8.423201558964254e11	0.07549379969947623
16	15.946286716607972	1.5514978880494067e12	1.570728736625802e12	0.05371328339202819
17	17.025427146237412	3.694735918486229e12	3.3169782238892363e12	0.025427146237412046
18	17.99092135271648	7.650109016515867e12	6.34485314179128e12	0.009078647283519814
19	19.00190981829944	1.1435273749721195e13	1.228571736671966e13	0.0019098182994383706
20	19.999809291236637	2.7924106393680727e13	2.318309535271638e13	0.00019070876336257925

Tabela 5: Wyniki eksperymentu w języku Julia

5.5.1 Wnioski

Rozbieżności pomiędzy faktycznymi pierwiastkami a wyliczonymi wynikają z ograniczeń precyzyjnych, które są związane z reprezentacją liczb zmiennoprzecinkowych w komputerze, a konkretnie z typem `Float64` używanym w języku Julia. Typ `Float64` (czyli podwójna precyzja zmiennoprzecinkowa) przechowuje liczby w formacie IEEE 754, co oznacza, że liczba jest reprezentowana przez 64 bity, z czego 52 bity są przeznaczone na część ułamkową (mantysę). Z tego powodu `Float64` może przechowywać tylko około 15 do 17 cyfr znaczących w systemie dziesiętnym.

W praktyce oznacza to, że liczby, które mają więcej niż 15-17 cyfr znaczących, mogą być zaokrąglane, co prowadzi do błędów reprezentacji. Takie błędy zaokrąglenia są szczególnie widoczne w zadaniach numerycznych związanych z wielomianami o dużych stopniach i dużych współczynnikach, jak ma to miejsce w przypadku wielomianu Wilkinsona.

Zatem obliczenie pierwiastków numerycznie za pomocą funkcji takich jak `roots` w Julii jest obciążone ryzykiem, że pierwiastki zostaną wyliczone z błędami wynikającymi z ograniczonej precyzji zmiennoprzecinkowej. W efekcie różnice

między pierwiastkami dokładnymi a wyliczonymi mogą wynosić nawet kilka jednostek w ostatnich miejscach po przecinku, szczególnie przy obliczaniu pierwiastków o dużych wartościach (np. $x = 20$).

5.6 Eksperymentalne badanie zmienionego wielomianu Wilkinsona

Aby dokładniej zbadać problem, w języku programowania Julia zaimplementowano algorytmy, które umożliwiają szczegółową analizę właściwości zmienionego wielomianu Wilkinsona.

k	z_k	$ \hat{P}(z_k) $	$ z_k - k $	
1.0 + 0.0im	0.9999999999996989	35696.50964788257	36626.425482422805	3.0109248427834245e-13
2.0 + 0.0im	2.0000000000283182	176252.60026668405	181303.93367257662	2.8318236644508943e-11
3.0 + 0.0im	2.9999999995920965	279157.6968824087	290172.2858891686	4.0790348876384996e-10
4.0 + 0.0im	3.9999999837375317	3.0271092988991085e6	2.0415372902750901e6	1.626246826091915e-8
5.0 + 0.0im	5.000000665769791	2.2917473756567076e7	2.0894625006962188e7	6.657697912970661e-7
6.00001 + 0.0im	5.999989245824773	1.2902417284205095e8	1.1250484577562995e8	2.0754175226400662e-5
6.9997 + 0.0im	7.000102002793008	4.805112754602064e8	4.572908642730946e8	0.0004020027930078385
8.00727 + 0.0im	7.999355829607762	1.6379520218961136e9	1.5556459377357383e9	0.007914170392238518
8.91725 + 0.0im	9.002915294362053	4.877071372550003e9	4.687816175648389e9	0.0856652943620535
10.09527 + 0.6435im	9.990413042481725	1.3638638195458128e10	1.2634601896949205e10	0.6519871406247129
10.09527 - 0.6435im	11.025022932909318	3.585631295130865e10	3.300128474498415e10	1.130722232139034
11.79363 + 1.65233im	11.953283253846857	7.533332360358197e10	7.388525665404988e10	1.660025177629511
11.79363 - 1.65233im	13.07431403244734	1.9605988124330817e11	1.8476215093144193e11	2.0905372562730324
13.99236 + 2.51883im	13.914755591802127	3.5751347823104315e11	3.5514277528420844e11	2.5200252008802893
13.99236 - 2.51883im	15.075493799699476	8.21627123645597e11	8.423201558964254e11	2.741839418520243
16.73074 + 2.81262im	15.946286716607972	1.5514978880494067e12	1.570728736625802e12	2.9199654481216957
16.73074 - 2.81262im	17.025427146237412	3.694735918486229e12	3.3169782238892363e12	2.828015519504366
19.50244 + 1.94033im	17.99092135271648	7.650109016515867e12	6.34485314179128e12	2.4595871869047055
19.50244 - 1.94033im	19.00190981829944	1.1435273749721195e13	1.228571736671966e13	2.0038490391477093
20.84691 + 0.0im	19.999809291236637	2.7924106393680727e13	2.318309535271638e13	0.8471007087633637

Tabela 6: Wyniki eksperymentu Julia

5.6.1 Wnioski

Eksperymenty potwierdziły wcześniejsze przypuszczenia, że wielomian Wilkinsona jest zadaniem źle uwarunkowanym. W miarę jak wartość pierwiastka rośnie, odchylenie od jego rzeczywistej wartości (czyli wartości 0 wielomianu dla danego pierwiastka) zwiększa się w sposób dramatyczny.

5.7 Uwarunkowanie i wpływ perturbacji

Z powyższego wyniku, że zmiana współczynnika w wielomianie (zaburzenie o wielkości $\epsilon = 2^{-23}$) powoduje znaczną zmianę pierwiastka $r = 20$. Przybliżony wynik $h \approx 102.76$ oznacza, że pierwiastek zostaje przesunięty o około 102 jednostki, co wskazuje na źle uwarunkowanie tego zadania numerycznego. Nawet bardzo małe zaburzenia współczynników mogą prowadzić do dużych zmian w pierwiastkach.

6 Model logistyczny

W niniejszym zadaniu rozważono model logistyczny opisujący wzrost populacji, który jest określony przez rekurencję:

$$p_{n+1} := p_n + rp_n(1 - p_n)$$

gdzie p_n jest proporcją populacji w n -tym kroku, a r jest współczynnikiem wzrostu. Początkowa wartość p_0 stanowi początkowy stan populacji. Model ten jest stosowany w badaniach nad dynamiką populacji w ekosystemach, gdzie wzrost populacji jest ograniczony przez pojemność środowiska (czynnik $(1 - p_n)$).

Celem eksperymentu było zbadanie wpływu różnych rodzajów arytmetyki (Float32 oraz Float64) na wyniki uzyskiwane przy 40 iteracjach powyższego modelu, jak również efekt wprowadzenia modyfikacji obcinania wyniku do trzech miejsc po przecinku po 10. iteracji.

6.1 Implementacja

Implementację modelu logistycznego wykonano w języku Julia, wykorzystując dwie funkcje:

- `rec(x, r, type)`: Funkcja ta realizuje obliczenie kolejnego elementu ciągu rekurencyjnego na podstawie aktualnej wartości p_n , współczynnika r , oraz wybranego typu arytmetycznego (Float32 lub Float64).
- `truncate(x, n)`: Funkcja służąca do obcięcia wyniku obliczeń do określonej liczby miejsc po przecinku. W eksperymencie zastosowano obcięcie do trzech miejsc po przecinku po 10. iteracji.

6.2 Eksperymenty

Dwa eksperymenty zostały zaplanowane:

1. **Test 1:** Porównanie wyników uzyskanych w arytmetyce Float32, z obcięciem do trzech miejsc po przecinku po 10 iteracjach.
2. **Test 2:** Porównanie wyników uzyskanych w arytmetyce Float32 i Float64 bez żadnej modyfikacji.

Obliczenia były przeprowadzane w oparciu o początkową wartość $p_0 = 0.01$ oraz współczynnik $r = 3$, a wyniki zostały przedstawione w postaci tabel.

Iteracja	Wynik z obciążeniem (Float32)	Wynik bez obciążenia (Float32)
1	0.0397	0.0397
2	0.15407173	0.15407173
3	0.5450726	0.5450726
4	1.2889781	1.2889781
5	0.1715188	0.1715188
6	0.5978191	0.5978191
7	1.3191134	1.3191134
8	0.056273222	0.056273222
9	0.21559286	0.21559286
10	0.722	0.7229306
11	1.3241479	1.3238364
12	0.036488414	0.037716985
13	0.14195944	0.14660022
14	0.50738037	0.521926
15	1.2572169	1.2704837
16	0.28708452	0.2395482
17	0.9010855	0.7860428
18	1.1684768	1.2905813
19	0.577893	0.16552472
20	1.3096911	0.5799036
21	0.09289217	1.3107498
22	0.34568182	0.088804245
23	1.0242395	0.3315584
24	0.94975823	0.9964407
25	1.0929108	1.0070806
26	0.7882812	0.9856885
27	1.2889631	1.0280086
28	0.17157483	0.9416294
29	0.59798557	1.1065198
30	1.3191822	0.7529209
31	0.05600393	1.3110139
32	0.21460639	0.0877831
33	0.7202578	0.3280148
34	1.3247173	0.9892781
35	0.034241438	1.021099
36	0.13344833	0.95646656
37	0.48036796	1.0813814
38	1.2292118	0.81736827
39	0.3839622	1.2652004
40	1.093568	0.25860548

Tabela 7: Porównanie wyników z obciążeniem po 10 iteracjach w arytmetyce Float32

Iteracja	Wynik w Float32	Wynik w Float64
1	0.0397	0.0397
2	0.15407173	0.15407173000000002
3	0.5450726	0.5450726260444213
4	1.2889781	1.2889780011888006
5	0.1715188	0.17151914210917552
6	0.5978191	0.5978201201070994
7	1.3191134	1.3191137924137974
8	0.056273222	0.056271577646256565
9	0.21559286	0.21558683923263022
10	0.7229306	0.722914301179573
11	1.3238364	1.3238419441684408
12	0.037716985	0.03769529725473175
13	0.14660022	0.14651838271355924
14	0.521926	0.521670621435246
15	1.2704837	1.2702617739350768
16	0.2395482	0.24035217277824272
17	0.7860428	0.7881011902353041
18	1.2905813	1.2890943027903075
19	0.16552472	0.17108484670194324
20	0.5799036	0.5965293124946907
21	1.3107498	1.3185755879825978
22	0.088804245	0.058377608259430724
23	0.3315584	0.22328659759944824
24	0.9964407	0.7435756763951792
25	1.0070806	1.315588346001072
26	0.9856885	0.07003529560277899
27	1.0280086	0.26542635452061003
28	0.9416294	0.8503519690601384
29	1.1065198	1.2321124623871897
30	0.7529209	0.37414648963928676
31	1.3110139	1.0766291714289444
32	0.0877831	0.8291255674004515
33	0.3280148	1.2541546500504441
34	0.9892781	0.29790694147232066
35	1.021099	0.9253821285571046
36	0.95646656	1.1325322626697856
37	1.0813814	0.6822410727153098
38	0.81736827	1.3326056469620293
39	1.2652004	0.0029091569028512065
40	0.25860548	0.011611238029748606

Tabela 8: Porównanie wyników dla Float32 i Float64

6.2.1 Wnioski

W eksperymencie 1 zaobserwowano, że do 18. iteracji wartości p_n z obcięciem oraz bez obcięcia pozostają niemal identyczne, pomimo wcześniejszego zastosowania obcięcia. Jednakże, od iteracji 19, wartości zaczynają wykazywać naprzemienną różnicę: wynik uzyskany w iteracji i -tej w przypadku obcięcia, pojawia się w iteracji $(i + 1)$ -ej w przypadku wyniku bez obcięcia. Tego rodzaju rozbieżność sugeruje, że algorytm jest niestabilny, a różnice między wartościami rosną z powodu kumulacji błędów numerycznych, które przenoszą się na kolejne iteracje.

W eksperymencie 2, porównując działanie algorytmu w arytmetyce `Float32` oraz `Float64`, również występuje istotna rozbieżność wyników. Różnice te pojawiają się jednak później niż w eksperymencie 1, bo dopiero w okolicach iteracji 30.

Z uwagi na to, że algorytm jest wrażliwy na niewielkie błędy popełniane w trakcie obliczeń, można go uznać za niestabilny.

7 Zbiory Julii

Zbiory Julii są obiektami matematycznymi znajdującymi się na płaszczyźnie zespolonej, które pomagają zrozumieć, jak iteracje wielomianów, takich jak $f(z) = z^2 + c$ lub $f(z) = z^3 + c$, zachowują się w zależności od wybranego parametru c . Dla wybranej wartości c , zaczynamy od liczby zespolonej z i iteracyjnie obliczamy kolejne wartości, podstawiając wynik poprzedniego obliczenia jako nową wartość z . Na przykład dla funkcji $z^2 + c$ proces wygląda następująco:

$$z \rightarrow z^2 + c \rightarrow (z^2 + c)^2 + c \rightarrow \dots$$

W zależności od wartości początkowej z , ciąg ten może przyjmować jedno z dwóch zachowań:

1. **Ciąg nieograniczony:** Kolejne iteracje z oddalają się w nieskończoność.
2. **Ciąg ograniczony:** Kolejne iteracje z pozostają w pewnym ograniczonym obszarze płaszczyzny zespolonej.

Zbiór Julii dla danego parametru c to granica oddzielająca punkty o tych dwóch rodzajach zachowania. Można powiedzieć, że to granica między „zbiorami uciekinierów” (punkty, z których wartości rosną w nieskończoność) a „zbiorami więźniów” (punkty, z których wartości pozostają ograniczone).

Zbiory Julii są również fascynujące ze względu na swoje właściwości samopodobieństwa. Mimo że ich struktura nie przypomina klasycznego, liniowego samopodobieństwa, jak w trójkącie Sierpińskiego, wykazują one samopodobieństwo oparte na przekształceniach nieliniowych. Powoduje to, że różne fragmenty zbioru Julii wyglądają podobnie do całości, lecz w różnych skalach i kształtach,

tworząc skomplikowane, fraktalne wzory, które często są badane wizualnie przy pomocy komputerów.

Analiza zbiorów Julii może nam powiedzieć wiele o uwarunkowaniu zadania związanego z rekurencyjnym równaniem:

$$x_{n+1} = x_n^2 + c, \quad n = 0, 1, 2, \dots$$

w kontekście stabilności oraz zachowania wartości x_n dla dużych n .

7.1 Jak zbiór Julii odnosi się do równania rekurencyjnego?

- **Granica zbioru Julii** – zbiór Julii dla danej wartości parametru c stanowi granicę między wartościami początkowymi x_0 , które prowadzą do ograniczonych ciągów x_n , a wartościami początkowymi, które prowadzą do nieograniczonych ciągów.
 - Jeśli x_0 leży w zbiorze więźniów, to iteracje będą ograniczone i stabilne.
 - Jeśli x_0 leży poza zbiorem Julii, ciąg będzie dążył do nieskończoności, co wskazuje na brak stabilności.
- **Uwarunkowanie zależne od początkowej wartości x_0** – dla wartości x_0 bliskich granicy zbioru Julii drobne zmiany wartości początkowej mogą drastycznie wpłynąć na zachowanie iteracji, co wskazuje na **wrażliwość** zadania na zaburzenia danych początkowych.

7.2 Opis eksperymentu

Przeprowadziliśmy eksperymenty dla różnych wartości stałej c oraz początkowych wartości x_0 . Dla każdej z par (c, x_0) , wykonano 40 iteracji równania rekurencyjnego w języku programowania Julia, korzystając z arytmetyki zmienno-przecinkowej typu Float64. Po każdej iteracji obliczano wartość x_n zgodnie z równaniem rekurencyjnym. Celem było zaobserwowanie zachowań generowanych ciągów, w szczególności stabilności, cykliczności oraz chaotyczności.

Poniżej przedstawiono zestawy danych, dla których przeprowadzono eksperymenty:

1. $c = -2, x_0 = 1$
2. $c = -2, x_0 = 2$
3. $c = -2, x_0 = 1.9999999999999999$
4. $c = -1, x_0 = 1$
5. $c = -1, x_0 = -1$

6. $c = -1, x_0 = 0.75$

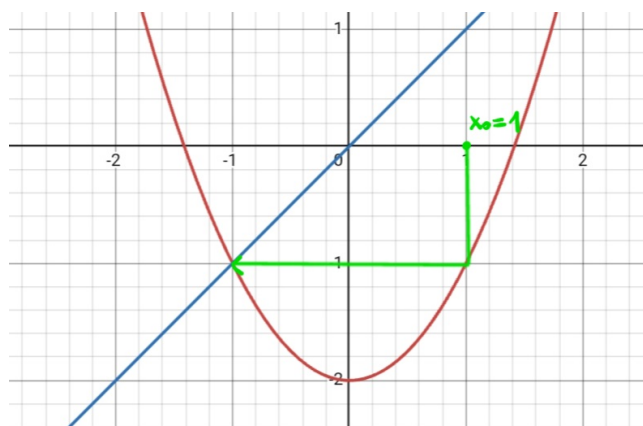
7. $c = -1, x_0 = 0.25$

7.3 Analiza wyników

7.3.1 Test1 $c = -2, x_0 = 1$

Iteracja	x
1	-1.0
2	-1.0
3	-1.0
4	-1.0
5	-1.0
6	-1.0
7	-1.0
8	-1.0
9	-1.0
10	-1.0
11	-1.0
12	-1.0
13	-1.0
14	-1.0
15	-1.0
16	-1.0
17	-1.0
18	-1.0
19	-1.0
20	-1.0
21	-1.0
22	-1.0
23	-1.0
24	-1.0
25	-1.0
26	-1.0
27	-1.0
28	-1.0
29	-1.0
30	-1.0
31	-1.0
32	-1.0
33	-1.0
34	-1.0
35	-1.0
36	-1.0
37	-1.0
38	-1.0
39	-1.0
40	-1.0

Tabela 9: Wartości ciągu dla $c = -2, x_0 = 1$



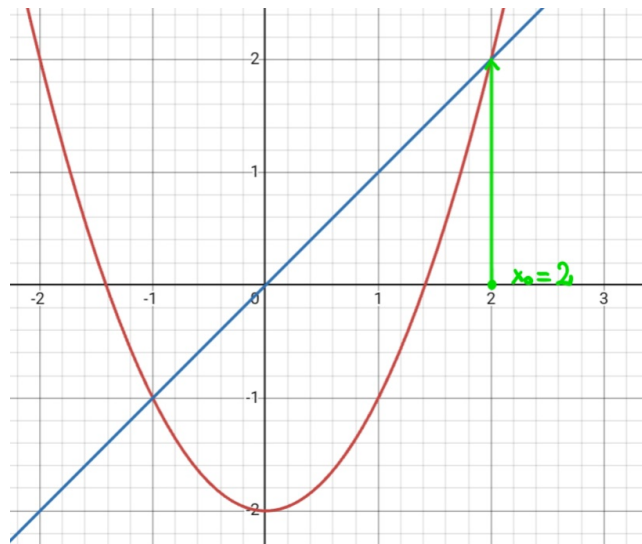
Rysunek 9: Wykres dla $c = -2, x_0 = 1$

Przy $c = -2$ i $x_0 = 1$ ciąg jest stałe równy -1 począwszy od pierwszej iteracji.

7.3.2 Test2 $c = -2, x_0 = 2$

Iteracja	x
1	2.0
2	2.0
3	2.0
4	2.0
5	2.0
6	2.0
7	2.0
8	2.0
9	2.0
10	2.0
11	2.0
12	2.0
13	2.0
14	2.0
15	2.0
16	2.0
17	2.0
18	2.0
19	2.0
20	2.0
21	2.0
22	2.0
23	2.0
24	2.0
25	2.0
26	2.0
27	2.0
28	2.0
29	2.0
30	2.0
31	2.0
32	2.0
33	2.0
34	2.0
35	2.0
36	2.0
37	2.0
38	2.0
39	2.0
40	2.0

Tabela 10: Wartości ciągu dla $c = -2, x_0 = 2$



Rysunek 10: Wykres dla $c = -2, x_0 = 2$

Przy $c = -2$ i $x_0 = 2$ ciąg jest stale równy 2 począwszy od pierwszej iteracji.

7.3.3 Test 3 $c = -2, x_0 = 1.999999999999999$

Iteracja	x
1	1.999999999999996
2	1.9999999999998401
3	1.9999999999993605
4	1.999999999997442
5	1.9999999999897682
6	1.999999999590727
7	1.99999999836291
8	1.999999993451638
9	1.999999973806553
10	1.99999989522621
11	1.999999580904841
12	1.999998323619383
13	1.999993294477814
14	1.999973177915749
15	1.9999892711734937
16	1.9999570848090826
17	1.999828341078044
18	1.9993133937789613
19	1.9972540465439481
20	1.9890237264361752
21	1.9562153843260486
22	1.82677862987391
23	1.3371201625639997
24	-0.21210967086482313
25	-1.9550094875256163
26	1.822062096315173
27	1.319910282828443
28	-0.2578368452837396
29	-1.9335201612141288
30	1.7385002138215109
31	1.0223829934574389
32	-0.9547330146890065
33	-1.0884848706628412
34	-0.8152006863380978
35	-1.3354478409938944
36	-0.21657906398474625
37	-1.953093509043491
38	1.8145742550678174
39	1.2926797271549244
40	-0.3289791230026702

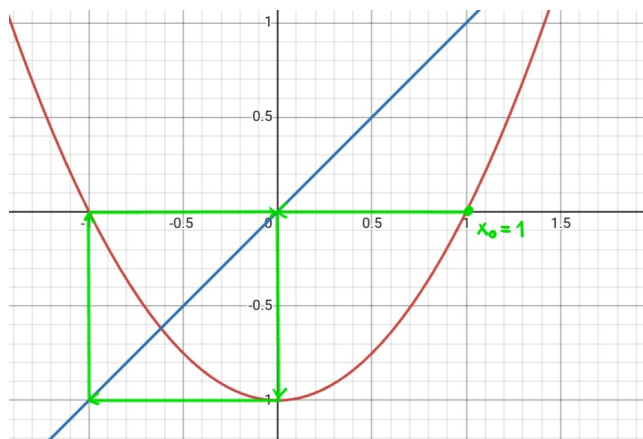
Tabela 11: Wartości ciągu dla $c = -2, x_0 = 1.999999999999999$

Przy $c = -2$ i $x_0 = 1.9999999999999999$ ciąg nie wydaje się zbiegać do żadnej wartości, ani stabilizować się wokół jakiegoś cyklu. Przez kilkanaście pierwszych iteracji wartości x_n pozostają blisko $x_0 = 1.9999999999999999$. Od około 19. iteracji wartości zaczynają się gwałtownie zmieniać i ostatecznie, przy 24. iteracji, zaczynają oscylować, z wartościami dodatnimi i ujemnymi w okolicach punktów oddalonych od początkowej wartości. Po 24. iteracji ciąg wykazuje niestabilność, w której wartości x_n dążą do coraz większych wartości bez żadnego ustabilizowanego wzorca, sugerując niestabilne zachowanie. Uzyskany efekt jest spowodowany zbyt małą próbą danych lub faktem, że punkt $x_0 = 1.9999999999999999$ nie leży w zbiorze Julii.

7.3.4 Test 4 $c = -1, x_0 = 1$

Iteracja	x
1	0.0
2	-1.0
3	0.0
4	-1.0
5	0.0
6	-1.0
7	0.0
8	-1.0
9	0.0
10	-1.0
11	0.0
12	-1.0
13	0.0
14	-1.0
15	0.0
16	-1.0
17	0.0
18	-1.0
19	0.0
20	-1.0
21	0.0
22	-1.0
23	0.0
24	-1.0
25	0.0
26	-1.0
27	0.0
28	-1.0
29	0.0
30	-1.0
31	0.0
32	-1.0
33	0.0
34	-1.0
35	0.0
36	-1.0
37	0.0
38	-1.0
39	0.0
40	-1.0

Tabela 12: Wartości ciągu dla $c = -1, x_0 = 1$



Rysunek 11: Wykres dla $c = -1, x_0 = 1$

Przy $c = -1$ i $x_0 = 1$, kolejne iteracje zachowują się w sposób **cykliczny**. Możemy sprawdzić, jak działa ten ciąg:

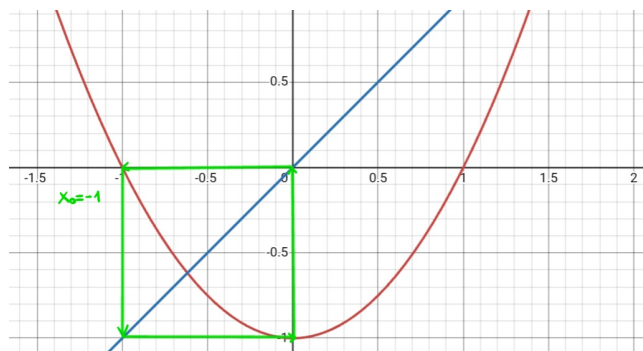
- **Krok 1:** $x_0 = 1$
- **Krok 2:** $x_1 = x_0^2 - 1 = 1^2 - 1 = 1 - 1 = 0$
- **Krok 3:** $x_2 = x_1^2 - 1 = 0^2 - 1 = 0 - 1 = -1$
- **Krok 4:** $x_3 = x_2^2 - 1 = (-1)^2 - 1 = 1 - 1 = 0$
- **Krok 5:** $x_4 = x_3^2 - 1 = 0^2 - 1 = -1$

Widzimy, że ciąg zaczyna cykl $1, 0, -1, 0, -1, \dots$

7.3.5 Test 5 $c = -1, x_0 = -1$

Iteracja	x
1	0.0
2	-1.0
3	0.0
4	-1.0
5	0.0
6	-1.0
7	0.0
8	-1.0
9	0.0
10	-1.0
11	0.0
12	-1.0
13	0.0
14	-1.0
15	0.0
16	-1.0
17	0.0
18	-1.0
19	0.0
20	-1.0
21	0.0
22	-1.0
23	0.0
24	-1.0
25	0.0
26	-1.0
27	0.0
28	-1.0
29	0.0
30	-1.0
31	0.0
32	-1.0
33	0.0
34	-1.0
35	0.0
36	-1.0
37	0.0
38	-1.0
39	0.0
40	-1.0

Tabela 13: Wartości ciągu dla $c = -1, x_0 = -1$



Rysunek 12: Wykres dla $c = -1, x_0 = -1$

Przy $c = -1$ i $x_0 = -1$, kolejne iteracje zachowują się w sposób **cykliczny**. Możemy sprawdzić, jak działa ten ciąg:

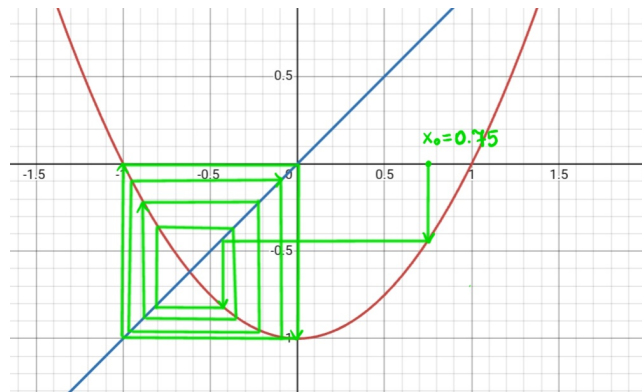
- **Krok 1:** $x_0 = -1$
- **Krok 2:** $x_1 = x_0^2 - 1 = (-1)^2 - 1 = 1 - 1 = 0$
- **Krok 3:** $x_2 = x_1^2 - 1 = 0^2 - 1 = -1$
- **Krok 4:** $x_3 = x_2^2 - 1 = (-1)^2 - 1 = 1 - 1 = 0$

Widzimy, że ciąg zaczyna cykl $1, 0, -1, 0, -1, \dots$

7.3.6 Test 6 $c = -1, x_0 = 0.75$

Iteracja	x
1	-0.4375
2	-0.80859375
3	-0.3461761474609375
4	-0.8801620749291033
5	-0.2253147218564956
6	-0.9492332761147301
7	-0.0989561875164966
8	-0.9902076729521999
9	-0.01948876442658909
10	-0.999620188061125
11	-0.0007594796206411569
12	-0.9999994231907058
13	-1.1536182557003727e-6
14	-0.9999999999986692
15	-2.6616486792363503e-12
16	-1.0
17	0.0
18	-1.0
19	0.0
20	-1.0
21	0.0
22	-1.0
23	0.0
24	-1.0
25	0.0
26	-1.0
27	0.0
28	-1.0
29	0.0
30	-1.0
31	0.0
32	-1.0
33	0.0
34	-1.0
35	0.0
36	-1.0
37	0.0
38	-1.0
39	0.0
40	-1.0

Tabela 14: Wartości ciągu dla $c = -1, x_0 = 0.75$



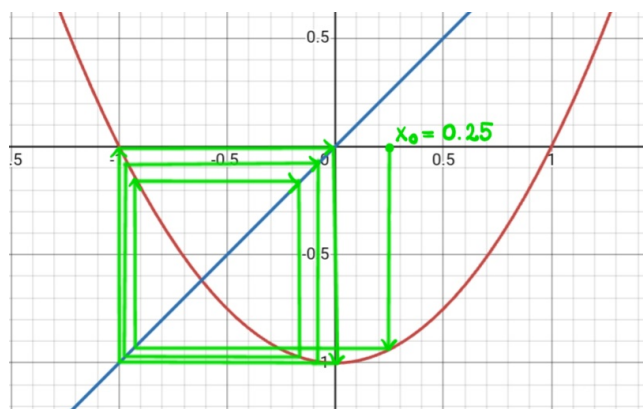
Rysunek 13: Wykres dla $c = -1, x_0 = 0.75$

Przy $c = -1$ i $x_0 = 0.75$ ciąg z początku wydaje się zachowywać w sposób chaotyczny, jednak od szesnastej iteracji stabilizuje się i zaczyna cykl $1, 0, -1, 0, -1, \dots$

7.3.7 Test 7 $c = -1, x = 0.25$

Iteracja	x
1	-0.9375
2	-0.12109375
3	-0.9853363037109375
4	-0.029112368589267135
5	-0.9991524699951226
6	-0.0016943417026455965
7	-0.9999971292061947
8	-5.741579369278327e-6
9	-0.9999999999670343
10	-6.593148249578462e-11
11	-1.0
12	0.0
13	-1.0
14	0.0
15	-1.0
16	0.0
17	-1.0
18	0.0
19	-1.0
20	0.0
21	-1.0
22	0.0
23	-1.0
24	0.0
25	-1.0
26	0.0
27	-1.0
28	0.0
29	-1.0
30	0.0
31	-1.0
32	0.0
33	-1.0
34	0.0
35	-1.0
36	0.0
37	-1.0
38	0.0
39	-1.0
40	0.0

Tabela 15: Wartości ciągu dla $c = -1, x_0 = 0.25$



Rysunek 14: Wykres dla $c = -1, x_0 = 0.25$

Przy $c = -1$ i $x_0 = 0.25$ ciąg z początku wydaje się zachowywać w sposób chaotyczny, jednak od jedenastej iteracji stabilizuje się i zaczyna cykl $1, 0, -1, 0, -1, \dots$

7.4 Wnioski

W trakcie przeprowadzonych przez nas eksperymentów pokazaliśmy, że dla testowanych par (c, x_0) jedynie para $(c = -2, x_0 = 1.9999999999999999)$ daje wyniki, które sugerują, że x_0 nie leży w zbiorze Julii.

Podsumowując, fakt, że tylko jedna para $c = -2$ i $x_0 = 1.9999999999999999$ nie leży w zbiorze Julii, może wskazywać na to, że algorytm działa prawidłowo, ale w okolicach granicy zbioru Julii może być bardzo wrażliwy na początkowe warunki, co jest naturalnym zjawiskiem w przypadku funkcji chaotycznych i układów nieliniowych.